

SensiSpace

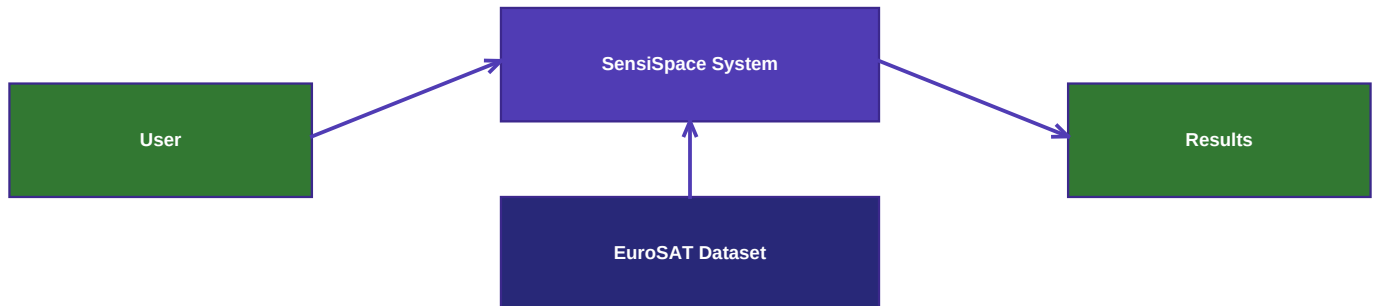
Data Flow Diagrams

Satellite Land Cover Classification System
Baby Dragon Hatchling (BDH) vs ResNet-18

BDH Architecture | ResNet-18 Pipeline | Training & Inference Flows

1. Level 0 - Context Diagram

The highest-level view showing SensiSpace as a single process boundary interacting with external entities - the User and the EuroSAT satellite dataset.



External Entities

- **User:** Uploads satellite images via web UI for land cover classification.
- **EuroSAT Dataset:** 27,000 Sentinel-2 satellite images across 10 land cover classes.

System Outputs

- **Classification Results:** BDH + ResNet predictions, confidence scores, spectral overlays, comparison metrics.

2. Level 1 - System Data Flow Diagram

Decomposition of the SensiSpace system into four major subsystems: Training Pipeline, Inference Engine, Spectral Segmentation, and Frontend Visualization.

Subsystems

#	Subsystem	Description
1.0	Training Pipeline	Loads EuroSAT, augments data, trains BDH & ResNet, saves weights + metrics
2.0	Inference & Prediction	Accepts uploaded image, runs dual-model forward pass, assembles results
3.0	Spectral Segmentation	HSV/LAB pixel analysis to generate land cover overlay visualization
4.0	Frontend Visualization	Renders results in tabbed UI with charts, comparison, and overlays

Data Stores

ID	Store	Contents
D1	saved_models/	resnet.pth, baby_dragon.pth (trained model weights)
D2	classes.json	List of 10 EuroSAT class names
D3	metrics.json	Accuracy, precision, recall, F1, inference time, params per model
D4	colors.json	RGB color map for each land cover class

Data Flows Between Subsystems

- EuroSAT Dataset --> [1.0 Training] --> D1 (weights), D2 (classes), D3 (metrics), D4 (colors)
- User Image --> [2.0 Inference] --> loads from D1, D2, D4
- [2.0 Inference] --> predicted class + probabilities --> [3.0 Spectral Segmentation]
- [3.0 Spectral] --> base64 overlay image --> [2.0 Inference] --> JSON response
- JSON response --> [4.0 Frontend] --> rendered UI to User
- D3 (metrics) --> [4.0 Frontend] --> benchmark comparison table

3. BDH Architecture - Data Flow Diagram

The Baby Dragon Hatchling (BDH) is a novel biologically-inspired architecture with four key innovations: Multi-Scale Feature Pyramids, Lateral Inhibition, Contextual Attention, and Recurrent Refinement.

3.1 End-to-End Pipeline

```

Input Image (3x64x64)
  |
  v
[STEM] Conv2d 3->32 + BN + ReLU -> Conv2d 32->64 + BN + ReLU
  |
  v
[BDH STAGE 1] MultiScale(64) -> LateralInhibition(64) -> ContextualAttention(64) + Residual
  | MaxPool2d (64ch, 32x32)
  v
[BDH STAGE 2] MultiScale(128) -> LateralInhibition(128) -> ContextualAttention(128) + Residual
  | MaxPool2d (128ch, 16x16)          <-- Grad-CAM target layer
  v
[BDH STAGE 3] MultiScale(256) -> LateralInhibition(256) -> ContextualAttention(256) + Residual
  | MaxPool2d (256ch, 8x8)
  v
[RECURRENT REFINEMENT] x2 iterations: Conv->BN->ReLU -> Gate(sigmoid) -> Gated Mix
  |
  v
[CLASSIFIER] GlobalAvgPool -> Dropout(0.3) -> FC(256->128) -> ReLU -> Dropout(0.2) -> FC(128->10)
  |
  v
Output: 10 Class Logits -> Softmax -> Predicted Land Cover

```

3.2 Multi-Scale Feature Pyramid Block

Processes input through three parallel convolution branches with different kernel sizes to capture features at multiple spatial scales - essential for satellite imagery where objects vary dramatically in size.

```

Input Feature Map
  |-----|-----| (parallel branches)
  v         v         v
[3x3 Conv] [5x5 Conv] [7x7 Conv]
[BatchNorm] [BatchNorm] [BatchNorm]
[ReLU]      [ReLU]      [ReLU]
  |          |          |
  v          v          v
  |--- Concatenate (channel dim) ---|
  |
  v
Multi-Scale Output

```

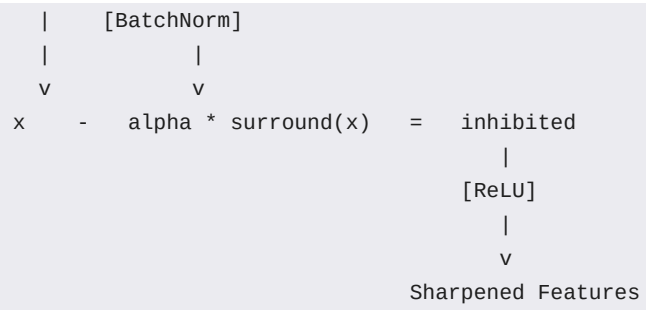
3.3 Lateral Inhibition Block

Biologically inspired by neural circuits where neighboring neurons suppress each other. Uses depthwise convolution as surround suppression with a learnable inhibition strength α .

```

Input x
  |-----| (two paths)
  |         v
  | [Depthwise Conv 3x3] (surround estimation)

```



3.4 Contextual Attention Module

Dual attention mechanism combining channel attention (Squeeze-and-Excitation style) and spatial attention to model long-range contextual relationships in geospatial imagery.

```

Input Features
|
v
[CHANNEL ATTENTION]
GlobalAvgPool -> FC(C -> C/8) -> ReLU -> FC(C/8 -> C) -> Sigmoid
--> Multiply channel weights with features
|
v
[SPATIAL ATTENTION]
AvgPool(channel) + MaxPool(channel) -> Concat -> Conv2d(7x7) -> Sigmoid
--> Multiply spatial weights with features
|
v
Attention-Weighted Features

```

3.5 Recurrent Refinement Module

A lightweight recurrent loop (2 iterations) that mimics the brain's top-down feedback pathways. Uses gated residual mixing to iteratively refine feature representations.

```

Input x (from Stage 3)
|  FOR i = 1 to 2:
|    refined = Conv2d(256->256) + BN + ReLU
|    gate    = Sigmoid(Conv2d 1x1)
|    x      = x * gate + refined * (1 - gate)
|  END FOR
|
v
Refined Features (256ch, 8x8)

```

4. ResNet-18 Architecture - Data Flow Diagram

Standard ResNet-18 with ImageNet pre-trained weights. The convolutional backbone is frozen (transfer learning); only the final fully connected layer is fine-tuned for 10 EuroSAT classes.

4.1 End-to-End Pipeline

```

Input Image (3x64x64)
  |
  v
[FROZEN BACKBONE - ImageNet Pre-trained]
  Conv2d(3->64, 7x7, stride=2) + BN + ReLU + MaxPool
  |
  v
Layer 1: 2x BasicBlock (64 channels)
  |
  v
Layer 2: 2x BasicBlock (128 channels)
  |
  v
Layer 3: 2x BasicBlock (256 channels)    <-- Grad-CAM target layer
  |
  v
Layer 4: 2x BasicBlock (512 channels)
  |
  v
AdaptiveAvgPool2d -> Flatten (512)
  |
  v
[TRAINABLE] Linear(512 -> 10)
  |
  v
Output: 10 Class Logits -> Softmax -> Predicted Land Cover

```

4.2 BasicBlock Internal Flow

```

Input x
  |-----| (residual skip)
  v       |
[3x3 Conv+BN] |
[ReLU]       |
[3x3 Conv+BN] |
  |           |
  v           v
+--- ADD ---+
  |
  [ReLU]
  |
  v
Output

```

4.3 Transfer Learning Strategy

- All convolutional parameters are FROZEN (requires_grad=False)
- Only the final FC layer (512 -> 10) is trainable
- Leverages ImageNet features for general visual recognition

- Represents industry-standard baseline approach

5. Training Pipeline - Data Flow Diagram

End-to-end training data flow from raw EuroSAT dataset to saved model weights and evaluation metrics.

5.1 Data Preparation Flow

```

EuroSAT Dataset (27,000 images, 10 classes)
  |
  v
[torchvision.datasets.EuroSAT] download=True
  |
  v
[random_split] 80% Train (21,600) / 20% Test (5,400)
  |           |
  v           v
[Train Augmentation]   [Test Transform]
  Resize 64x64         Resize 64x64
  RandomHFlip          ToTensor
  RandomVFlip          Normalize
  RandomRotation(15)
  ColorJitter
  ToTensor
  Normalize
  |           |
  v           v
[DataLoader batch=32]  [DataLoader batch=32]

```

5.2 Training Loop Flow

```

FOR model IN [ResNet-18, BDH]:
  Initialize model (pretrained or from scratch)
  optimizer = Adam(lr=1e-3)
  scheduler = CosineAnnealingLR(T_max=10)
  criterion = CrossEntropyLoss
  |
  FOR epoch = 1 to 10:
    FOR batch IN train_loader:
      images, labels --> model --> outputs
      loss = criterion(outputs, labels)
      loss.backward() --> optimizer.step()
    END
    scheduler.step()
  END
  |
  v
[Evaluate on test_loader]
  accuracy, precision, recall, F1, inference_time
  |
  v
[Save] model.pth, classes.json, metrics.json, colors.json

```

6. Inference & Prediction - Data Flow Diagram

Real-time prediction data flow when a user uploads a satellite image through the web interface.

6.1 Request Flow

```

User uploads image via browser
  |
  v
[Frontend] FileReader -> preview -> POST /api/predict (FormData)
  |
  v
[FastAPI] Read UploadFile bytes
  |
  v
[PIL] Image.open().convert("RGB")
  |
  v
[Transform] Resize(64x64) -> ToTensor -> Normalize -> unsqueeze(0)
  |
  |--- Tensor cloned for each model ---|
  v                                     v
[ResNet-18 Forward Pass]             [BDH Forward Pass]
  torch.no_grad()                     torch.no_grad()
  output -> softmax -> probs           output -> softmax -> probs
  argmax -> predicted class             argmax -> predicted class
  measure inference time                measure inference time
  |                                     |
  v                                     v
[Spectral Segmentation]               [Spectral Segmentation]
  generate_cam_image()                 generate_cam_image()
  -> base64 overlay                    -> base64 overlay
  |                                     |
  v                                     v
  |----- Assemble JSON Response -----|
  |
  v
{ "resnet": {prediction, confidence, all_probs, cam_image, ...},
  "baby_dragon": {prediction, confidence, all_probs, cam_image, ...} }
  |
  v
[Frontend] renderResults() -> Tabs + Charts + Comparison

```

7. Spectral Segmentation - Data Flow Diagram

Pixel-accurate land cover visualization using HSV/LAB color-space analysis instead of Grad-CAM. Produces precise boundaries by analyzing actual pixel colors.

7.1 Segmentation Pipeline

```

Original PIL Image + Model Class Probabilities
|
v
[Resize to 400x400] -> RGB numpy array
|
|--- cv2.cvtColor ---|--- cv2.cvtColor ---|
v             v             v
[RGB Channels]   [HSV: H,S,V]   [LAB: L,A,B]
R, G, B
|
v
[Vegetation Indices]
GLI = (2G - R - B) / (2G + R + B + 1)
ExG = 2G - R - B
|
v
[Generate 6 Spectral Masks]
Dense Vegetation:  GLI>0.05, S>40, green hue
Light Vegetation:  near-zero GLI, moderate sat
Water:             blue hue OR dark+blue tint
Urban:             low sat, warm tones
Road:             very low sat, neutral LAB
Bare Soil:         brownish, warm hue
|
v
[Morphological Cleanup]
Close (5x5 ellipse) + Open (3x3 ellipse)
|
v
[Overlap Resolution] Priority: Water > Vegetation > Urban > Road > Soil
|
v
[Map Spectral -> EuroSAT Classes]
dense_veg -> Forest, HerbaceousVegetation, Pasture
water      -> River, SeaLake
urban      -> Residential, Industrial
Pick candidate with highest model probability
|
v
[Overlay Rendering]
Color fill per region (alpha = prob * 0.4)
Draw contours (cv2.findContours)
Add text labels: class_name + confidence%
|
v
Base64 JPEG encoded overlay image

```

8. BDH vs ResNet-18 - Data Flow Comparison

Aspect	BDH (Baby Dragon Hatchling)	ResNet-18
Input Shape	3 x 64 x 64	3 x 64 x 64
Feature Extraction	Multi-Scale Pyramid (3x3+5x5+7x7)	Sequential BasicBlocks (3x3+3x3)
Contrast Enhancement	Lateral Inhibition (surround suppression)	None
Attention	Dual Channel + Spatial Attention	None
Feedback Processing	Recurrent Refinement (2 gated iters)	None (pure feedforward)
Pretrained Weights	Trained from scratch	ImageNet pretrained (frozen)
Trainable Layers	All layers	Only final FC layer
Classifier	256->128->10 (with dropout)	512->10 (single linear)
Grad-CAM Target	stage2 (16x16 activations)	layer3 (8x8 activations)
Output	10-class logits -> Softmax	10-class logits -> Softmax

Key Architectural Differences

- **Multi-Scale vs Sequential:** BDH captures features at 3 scales simultaneously; ResNet processes sequentially with fixed 3x3 kernels.
- **Biological Inspiration:** BDH uses lateral inhibition (neural suppression) and recurrent refinement (feedback loops) not found in ResNet.
- **Attention Mechanisms:** BDH applies dual channel+spatial attention at every stage; ResNet has no attention.
- **Transfer vs Novel:** ResNet leverages ImageNet knowledge; BDH learns satellite-specific features from scratch.
- **Parameter Efficiency:** BDH has fewer parameters but compensates with architectural innovations.